

Learn Python for Automation

Lesson 3: Control Flow: Conditions & Loops

Control Flow: Conditions & Loops - Study Notes

Key Concepts

- **Conditional Statements** (`if`, `elif`, `else`) decide which block of code runs based on Boolean expressions.
- **For Loops** iterate over an iterable (lists, strings, ranges) with a fixed number of iterations.
- **While Loops** run as long as a condition remains true, allowing dynamic iteration counts.
- **Break & Continue** alter loop flow: `break` exits the loop immediately, `continue` skips to the next iteration.
- **Indentation** defines the block of code that belongs to each control structure; it is syntactically significant in Python.

Summary

Control flow lets a program decide what to do next, whether to repeat actions, or skip parts of code. Conditional statements (`if`, `elif`, `else`) let you branch execution based on conditions. Loops (`for` and `while`) enable repetition: `for` is ideal when the number of iterations is known or bounded by an iterable, while `while` is used when the loop should continue until a condition changes. Within loops, `break` and `continue` provide fine grained control: `break` stops the loop entirely, whereas `continue` skips the remaining code in the current iteration and proceeds to the next one. Mastering these constructs is essential for writing efficient, readable, and dynamic Python programs.

Important Points

- **Boolean Expressions**: The condition in `if`/`elif`/`while` must evaluate to `True` or `False`.
- **`elif` vs. `else`**: Use `elif` when you have multiple distinct conditions; `else` catches everything else.
- **For loop Syntax**: `for variable in iterable:` – the loop variable takes each element in turn.
- **Range Function**: `range(start, stop, step)` is commonly used with `for` to generate numeric sequences.
- **While loop Pitfall**: Ensure the loop condition eventually becomes `False`; otherwise, you'll create an infinite loop.
- **Break vs. Continue**:
 - `break` ! exits the nearest enclosing loop.
 - `continue` ! skips the rest of the loop body for the current iteration.
- **Nested Loops**: Loops can be placed inside other loops; `break`/`continue` affect only the innermost loop unless specified otherwise.
- **Indentation**: In Python, indentation defines block scope; mismatched indentation results in `IndentationError`.

Examples

1. Conditional Logic – Grading

```
```python
score = 85
if score >= 90:
 grade = 'A'
elif score >= 80:
 grade = 'B'
elif score >= 70:
 grade = 'C'
elif score >= 60:
 grade = 'D'
else:
 grade = 'F'
print(f"Score: {score} !' Grade: {grade}")
```
```

Explanation:

- The program checks each condition in order.
- Once a condition is `True`, the corresponding block runs and the rest is skipped.
- If none of the `if/elif` conditions match, the `else` block executes.

2. Loop with Break & Continue – Prime Number Finder

```
```python
def is_prime(n):
 if n < 2:
 return False
 for i in range(2, int(n**0.5) + 1):
 if n % i == 0: # Found a divisor !' not prime
 return False
 return True
```

## Find first 10 prime numbers

```
primes_found = 0
candidate = 2
while primes_found < 10:
 if is_prime(candidate):
 print(candidate, end=' ')
 primes_found += 1
 candidate += 1 # Move to next number
```
```

Explanation:

- The ``while`` loop continues until 10 primes are found.
- ``continue`` is not used here, but you could use it inside ``is_prime`` to skip the rest of the loop body when a divisor is found.
- ``break`` could be used to exit the loop immediately once a condition is met (e.g., if a search limit is reached).

Quick Review

1. ****What is the purpose of the ``elif`` keyword?****
2. ****How does a ``for`` loop differ from a ``while`` loop?****
3. ****What happens when ``break`` is executed inside a nested loop?****
4. ****Write a simple ``while`` loop that prints numbers 1 to 5.****
5. ****Explain the difference between ``break`` and ``continue``.****

Further Reading

- ****List Comprehensions**** – concise way to create lists using loops.
- ****Generator Expressions**** – lazy evaluation of iterables.
- ****Exception Handling**** (``try``, ``except``, ``finally``) – controlling flow when errors occur.
- ****Recursion**** – using function calls to repeat logic.
- ****Functional Programming Constructs**** (``map``, ``filter``, ``reduce``) – alternative to explicit loops.

